

Question Answering Pipeline

Roberto Herrera, Bach Nguyen
Department of Electrical Engineering and Computer Science
University of Missouri

CS 8001 - Designing End-to-End ML Systems
Project Milestone 1

October 7, 2026

1 Overview

This report describes a two stage question answering pipeline. The objective is to answer questions about long documents with small extractive QA models from the BERT family, which can run on edge devices. These models read at most 512 tokens, roughly 2800 characters, with the question sharing that window with the text. Hence, we need to reduce each context before the QA model runs. In the first stage of this pipeline, we split the context into chunks, embeds the chunks with a sentence transformer model, stores them in a ChromaDB vector store, and retrieve the chunks nearest to the question. Then, stage 2 reads the question together with the retrieved chunks, selects an answer span, and returns it with a confidence score. An assertion checks that score against a threshold. For every configuration we report accuracy and latency per question. We evaluate the pipeline on two datasets, HotpotQA and QASPER, and vary the chunking method, the search method, the embedding model and the QA model. The report describes the datasets, the two stages and the assertion, and then reports accuracy and latency for each configuration.

2 Datasets

We use two datasets, HotpotQA and QASPER.

HotpotQA [1] is a question answering dataset built from Wikipedia. Almost every question comes with ten paragraphs. Two of them hold the evidence, and the other eight are distractors, which are Wikipedia paragraphs that look related to the question but do not hold the answer. The file also lists the supporting facts, the sentences that hold the evidence. The training file has 90,447 examples. Retrieval must therefore find the two paragraphs that hold the evidence among the distractors. The average context is about 5,400 characters, roughly twice the limit of the QA models.

QASPER [2] is built from full NLP research papers, with evidence passages written by annotators. Answers can be extractive spans, free form text, yes or no, or unanswerable. The training file has 888 papers with a mean length of about 24,000 characters. On average a paper is eight to nine times longer than the 2800 character limit of the QA models, so chunking matters much more here than for HotpotQA.

3 Architecture

3.1 Stage 1, Retrieval

Stage 1 cuts the context into chunks and keeps only the chunks closest to the question. We cut the text with one of two methods. Recursive splitting breaks the text at paragraph, sentence and word boundaries until each chunk fits a size cap. Semantic splitting embeds each sentence and cuts where the next sentence is much less similar to the one before it. We tested caps of 64, 128 and 256 tokens.

Each chunk is embedded with one of three sentence transformer models, MiniLM L6, MiniLM L12 and mpnet. The chunks are stored in a ChromaDB vector store with cosine distance. HotpotQA gets a temporary store for each question, and QASPER gets one store for each paper, reused for all of its questions.

The question is embedded with the same model, and we use one of two searches. Single pass search returns the k nearest chunks. Two hop search takes the nearest chunk, adds its text to the question, searches again without the paragraph of that chunk, and fills the remaining places with the results of the second search. We use k of 1, 3, 5 and 10. The chunks are placed after the question in rank order, and the last chunk is trimmed so that everything fits the 512 tokens of the QA model. About 2100 characters of chunks reach the QA model.

3.2 Stage 2, Question Answering

Stage 2 reads the question together with the retrieved chunks, up to 512 tokens in total, and selects the answer from them. The answer is a span of words that already sits in the chunks, so the QA model selects that span, it does not write it. The QA model scores every token of the chunks twice, once as the start of the answer and once as the end. It returns the span between the best start and the best end, together with a confidence score.

We use three QA models from Hugging Face, tinyroberta-squad2, roberta-base-squad2 and bert-large-uncased-whole-word-masking-finetuned-squad. The first two were trained to include unanswerable questions, so they can return an empty answer. The third was trained without unanswerable questions, so it always returns a span.

3.3 Assertion and Metrics

The assertion passes when the confidence score of the QA model is at least 0.5 and the context is not empty, and the number of passed assertions is the accuracy count of a configuration. We report the number of passed assertions together with exact match and F1.

Neither dataset labels chunks. Both label the evidence, the paragraphs or passages that hold the answer. We therefore score each retrieved chunk through the evidence it comes from, and an evidence paragraph counts as found when any of its chunks is retrieved [3]. The evidence is only a small part of each context, so scores depend on how much text we retrieve. Accuracy credits a question as soon as any evidence is found, so missing evidence goes unseen. Recall counts the evidence found, and it rises with more or larger chunks, because they cover more of the context and catch more of the evidence. Precision counts the share of what we retrieve that is evidence, and it favors small chunks, because a large chunk brings unrelated text from the passages around the evidence. F1 balances the two, so it is high only when the evidence is found and little else is retrieved. We report recall, precision and F1 of the evidence at k of 1, 3, 5 and 10, on all retrieved chunks and on the chunks that fit the 512 token window. The window count shows how much of

the retrieved evidence the QA model sees, and the gap between the two counts is what a larger window would add.

We score answers with exact match and F1 [4]. Exact match counts only identical answers, and F1 also credits shared words, so a partly right answer earns partial credit. Latency per question is the time of retrieval, packing and QA, and the indexing time of the vector store is reported separately as an average per question. All times were measured on one laptop CPU.

3.4 Pipeline Configurations

The pipeline has five choices. Each combination of choices is one pipeline configuration, which gives 72 configurations.

Choice	Options	Count
Dataset	HotpotQA, QASPER	2
Chunking method	Recursive, semantic	2
Search method	Single pass, two hop	2
Embedding model	MiniLM L6, MiniLM L12, mpnet	3
QA model	tinyroberta, roberta base, bert large	3
Configurations	$2 \times 2 \times 2 \times 3 \times 3$	72

The chunk size and the number of retrieved chunks are settings inside each configuration. We did not run the full grid, because each question takes up to several seconds on a CPU. We chose 20 configurations, 10 for each dataset, with chunks of 128 tokens, since chunks of 64 tokens gave a lower F1 at k of 1 and 3 and chunks of 256 tokens gave no higher F1 on HotpotQA (Table 2). Each was run with k of 1, 3, 5 and 10. The baseline uses the simplest option for every choice, which is MiniLM L6, recursive chunking, single pass search and tinyroberta, and each other configuration changes one choice from it, namely semantic chunking, a larger embedding model, a larger QA model or two hop search, so that any change in the results comes from that choice. Two hop search was also run with semantic chunking and with each embedding model, and the QA models were compared on the retrieval of the baseline so that they receive the same context. Chunks of 64 and 256 tokens were run only with MiniLM L6, recursive chunking and tinyroberta. In total this gives 57 settings for each dataset on 157 questions, 100 from HotpotQA and 57 from QASPER, which is 8,949 evaluations.

4 Results

4.1 Experiments

We change one choice at a time from the baseline, so that any difference in the results can be attributed to that choice. The pipeline is meant to run on an edge device without a GPU, so we only consider small embedding models and QA models, and every experiment runs on a CPU. The experiments answer four questions. First, does two hop search retrieve better evidence than single pass search, and in which dataset? HotpotQA questions need evidence from two paragraphs, while QASPER questions are answered from one long paper, so the two datasets can respond differently. Second, do the embedding model and the chunking method place the needed evidence inside the 512 token window, which we measure with evidence recall after packing. Third, when the evidence is in the window, does the QA model have enough context to answer correctly, and how much does a larger QA model add on the same retrieved context? Fourth, what does each gain cost in latency?

4.2 Accuracy and Latency

Table 1 reports each of the 20 configurations at the number of retrieved chunks with the highest F1. With 100 questions from HotpotQA and 57 from QASPER, F1 differences below about 0.05 are within sampling noise. Table 2 shows the F1 of the baseline retrieval for each choice on the number of chunks.

Dataset	Chunking	Search	Embedder	QA model	k	F1	Assertions passed	Latency (s)
HotpotQA	Recursive	Single pass	MiniLM L6	bert large	10	0.544	51%	4.63
HotpotQA	Recursive	Single pass	MiniLM L6	roberta base	10	0.407	24%	1.11
HotpotQA	Recursive	Single pass	mpnet	tinyroberta	5	0.387	23%	0.52
HotpotQA	Recursive	Two hop	MiniLM L12	tinyroberta	5	0.374	20%	0.63
HotpotQA	Recursive	Two hop	MiniLM L6	tinyroberta	5	0.373	19%	0.50
HotpotQA	Recursive	Single pass	MiniLM L12	tinyroberta	5	0.349	20%	0.55
HotpotQA	Recursive	Two hop	mpnet	tinyroberta	5	0.338	18%	0.79
HotpotQA	Semantic	Single pass	MiniLM L6	tinyroberta	5	0.330	28%	0.41
HotpotQA	Semantic	Two hop	MiniLM L6	tinyroberta	5	0.330	28%	0.49
HotpotQA	Recursive	Single pass	MiniLM L6	tinyroberta	5	0.328	18%	0.44
QASPER	Semantic	Two hop	MiniLM L6	tinyroberta	3	0.188	58%	0.22
QASPER	Recursive	Two hop	MiniLM L6	tinyroberta	3	0.177	68%	0.32
QASPER	Recursive	Single pass	MiniLM L6	bert large	5	0.169	19%	3.87
QASPER	Recursive	Two hop	mpnet	tinyroberta	3	0.167	65%	0.40
QASPER	Recursive	Single pass	MiniLM L12	tinyroberta	3	0.165	68%	0.27
QASPER	Recursive	Single pass	MiniLM L6	tinyroberta	3	0.156	61%	0.25
QASPER	Recursive	Single pass	MiniLM L6	roberta base	3	0.152	42%	0.46
QASPER	Recursive	Two hop	MiniLM L12	tinyroberta	5	0.152	72%	0.48
QASPER	Semantic	Single pass	MiniLM L6	tinyroberta	5	0.150	68%	0.32
QASPER	Recursive	Single pass	mpnet	tinyroberta	5	0.150	74%	0.35

Table 1: Best result of each of the 20 configurations with chunks of 128 tokens. Each configuration is reported at the number of retrieved chunks k with the highest F1.

Dataset	QA model	Chunk size	$k = 1$	$k = 3$	$k = 5$	$k = 10$
HotpotQA	tinyroberta	64	0.223	0.251	0.300	0.276
HotpotQA	tinyroberta	128	0.286	0.312	0.328	0.318
HotpotQA	tinyroberta	256	0.285	0.302	0.322	0.316
HotpotQA	roberta base	128	0.277	0.404	0.400	0.407
HotpotQA	bert large	128	0.307	0.433	0.483	0.544
QASPER	tinyroberta	64	0.074	0.119	0.154	0.085
QASPER	tinyroberta	128	0.113	0.156	0.103	0.119
QASPER	tinyroberta	256	0.097	0.153	0.138	0.138
QASPER	roberta base	128	0.100	0.152	0.133	0.128
QASPER	bert large	128	0.102	0.148	0.169	0.155

Table 2: F1 by number of retrieved chunks k for the baseline retrieval, with chunks of 64, 128 and 256 tokens for tinyroberta and chunks of 128 tokens for the other QA models.

On HotpotQA, the QA model has the largest effect. Bert large reaches an F1 of 0.544 and passes the assertion for 51 of 100 questions, against 0.407 for roberta base and 0.387 for the best tinyroberta configuration, but it takes 4.63 seconds per question against 0.52 seconds. Its F1 grows from 0.307 at k of 1 to 0.544 at k of 10, while roberta base and tinyroberta stay almost flat after k of 3. Among the tinyroberta configurations, two hop search raises the F1 of MiniLM L6 from

0.328 to 0.373 and of MiniLM L12 from 0.349 to 0.374, but lowers it for mpnet from 0.387 to 0.338. Semantic chunking matches recursive chunking in F1, 0.330 against 0.328. For tinyroberta, chunks of 64 tokens give the lowest F1 on both datasets at k of 1 and 3.

QASPER scores are lower, from 0.150 to 0.188. The best configuration is tinyroberta with semantic chunking and two hop search at k of 3, with 0.188 at 0.22 seconds per question. The larger QA models bring no gain, and bert large reaches 0.169 at 3.87 seconds and passes only 19% of the assertions, against 58% to 74% for tinyroberta. For the baseline retrieval, the share of gold sentences in the window rises from 64% at k of 3 to 81% at k of 10 on HotpotQA, and from 26% to 31% on QASPER.

4.3 Findings

Two hop search helps the MiniLM models slightly and hurts mpnet, so we do not treat it as a reliable improvement. The embedding model and the chunking method move F1 by less than the sampling noise, so with the options we tested they are not what limits accuracy.

The limit differs by dataset. On HotpotQA, more than 80% of the gold sentences reach the window at k of 10, yet F1 rises from 0.318 with tinyroberta to 0.544 with bert large on the same context. The evidence is mostly there, so the QA model is the bottleneck, and a larger one costs 4.63 seconds per question against 0.5. Retrieval limits accuracy only at small k , and the 512 token window is full by k of 10, so no more chunks fit. On QASPER, only about a third of the gold sentences reach the window and a larger QA model does not help, so retrieval comes first. Better accuracy there would need better retrieval over long papers, and on a CPU edge device every gain has to be paid for in latency.

5 Reproducibility

All code is in one notebook in our GitHub repository, together with the Dockerfile. The container includes the notebook and downloads the datasets and models. All random choices use a fixed seed, so every configuration is evaluated on the same questions. We ran everything on one laptop with an Intel Core i5-1235U processor, 7.6 GB of memory for Docker and no GPU.

6 Data and Code Availability

The notebook, the Dockerfile, the results of every run and this report are in our GitHub repository at <https://github.com/Robertoherrera0/QA-retrieval-pipeline>.

7 Team Organization

RH built Stage 1, BN built Stage 2. We chose the best configuration together by joining the retrieval and QA results. We communicated through Microsoft Teams and shared documents for all the required material.

References

- [1] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. W. Cohen, R. Salakhutdinov, C. D. Manning. HotpotQA, a dataset for diverse, explainable multi-hop question answering. EMNLP 2018.

- [2] P. Dasigi, K. Lo, I. Beltagy, A. Cohan, N. A. Smith, M. Gardner. A dataset of information seeking questions and answers anchored in research papers. NAACL 2021.
- [3] R. Qu, R. Tu, F. Bao. Is semantic chunking worth the computational cost? Findings of NAACL 2025.
- [4] D. Yu. Yuvzhenko, S. H. Stirenko. A comparative study of chunking strategies for retrieval-augmented generation. Cybernetics and Systems Analysis 62(3):385–394, 2026.